```
])
```

produces:

```
You have three consecutive high cards
```

This works, because now the pattern match runs against the hash returned by deconstruct_keys.

The deconstruct_keys method takes an argument that in many cases you can ignore. The value of that argument is the set of keys that the pattern is actually inquiring about, and the purpose of that information is to allow you to send back a subset of your object for performance reasons. For simple cases, the subset operation is probably more expensive than just creating the hash. If the argument is nil, then the pattern has used ** to request the entire hash. The return value of deconstruct_keys is arbitrary, but typically it's a hash representation of the data in the object.

There is an analogous method called deconstruct which allows your class to match against array patterns. It takes no arguments, and the expectation is that you return an array that is a representation of your instance. We suspect that fewer classes will use deconstruct than deconstruct_keys—it seems like many classes don't have an array representation with a clear order. But if you do have a class where the data has a clearly ordered representation, like a date, or a cartesian point, or something, then deconstruct would be useful. A class can implement both if there is both a useful array and hash representation.

What's Next

In this chapter, we went through a lot of different Ruby expressions, from assignment to math to logic to loops to patterns. Next we'll look at Ruby's exception handling and see what to do when things go wrong.